Cybersecurity Detection Model — Research Prompt

Source prompt for an improved IDS machine learning study, extending El-Taie & Kraidi (2023), JCIM Vol.12.

Context From The Paper

I'm sharing a research paper titled "Cybersecurity Detection Model using Machine Learning Techniques" (El-Taie & Kraidi, 2023, JCIM Vol.12). The goal is to deeply understand its methodology, dataset, and results, and then build an IMPROVED machine learning model that outperforms the paper's benchmarks.

Dataset: NSL-KDD style Intrusion Detection System (IDS) dataset from Kaggle (a well-known public dataset — use the standard NSL-KDD train/test CSVs, or if unavailable, simulate a realistic synthetic dataset with the same 42 features and statistical distributions described below, so the code remains fully runnable).

- 42 features including: duration, protocol_type (tcp/udp/icmp), service (http/ftp_data/private/other/etc.), flag (SF/S0/REJ/etc.), src_bytes, dst_bytes, land, wrong_fragment, urgent, hot, num_failed_logins, logged_in, num_compromised, root_shell, su_attempted, num_root, num_file_creations, num_shells, num_access_files, num_outbound_cmds, is_host_login, is_guest_login, count, srv_count, serror_rate, srv_serror_rate, rerror_rate, srv_rerror_rate, same_srv_rate, diff_srv_rate, srv_diff_host_rate, dst_host_count, dst_host_srv_count, dst_host_same_srv_rate, dst_host_diff_srv_rate, dst_host_same_src_port_rate, dst_host_srv_diff_host_rate, dst_host_serror_rate, dst_host_srv_serror_rate, dst_host_rerror_rate, dst_host_srv_rerror_rate
- ~25,192 records
- Binary classification target: "normal" (0) vs "anomaly" (1)
- Class distribution: ~13,400 normal vs ~11,800 anomaly (mildly imbalanced, ~53/47%)
- Note: num_outbound_cmds and is_host_login have zero variance (constant columns) in the original data — handle/drop these appropriately

Preprocessing used in the paper (baseline to replicate first):

1. Label encoding of categorical features (protocol_type, service, flag) and target class

2. Feature selection via correlation heatmap — dropped weakly correlated/zero-variance features
3. No SMOTE/oversampling, no advanced scaling, no cross-validation mentioned

Baseline Results from the paper (THESE ARE THE TARGETS TO BEAT):

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	94.9%	94.1%	94.9%	95.2%
K-Nearest Neighbor	98.8%	98.5%	98.8%	98.9%
Random Forest	99.81%	99.80%	99.81%	99.82%

Specific Target For This Study

- **Primary goal:** Beat Random Forest's 99.81% accuracy and 99.82% F1-score using a tuned ensemble/boosting approach or stacked model.
- **Secondary goal:** Since this is an Intrusion Detection System, minimize **False Negatives** (missed attacks) specifically — i.e., prioritize **Recall** for the "anomaly" class even if it costs a very small amount of precision, since a missed cyberattack is more costly than a false alarm.
- Report a confusion matrix breakdown (TP/TN/FP/FN) for every model so the False Negative count is explicitly visible and compared across models.

Evaluation metric formulas used in the paper (keep consistent):

- Accuracy = (TP+TN)/(TP+TN+FP+FN)
- Precision = TP/(TP+FP)
- Recall = TP/(TP+FN)
- F1 = 2*(Precision*Recall)/(Precision+Recall)

Limitations in the paper (address these in the improved approach):

- Only 3 classical ML models — no boosting (XGBoost/LightGBM/CatBoost), no deep learning
- No hyperparameter tuning (likely default sklearn parameters)
- No cross-validation (risk of overfitting on a single train-test split)
- No class imbalance handling (SMOTE/class weighting)

- No feature importance analysis beyond a correlation heatmap
- No ROC-AUC or confusion matrix reported
- No training/inference time comparison

What To Do

1. **Replicate the baseline** — preprocess data the same way as the paper (encoding + correlation-based feature selection) and implement LR, KNN, RF with default/similar settings to reproduce comparable numbers.
2. **Build improved models**:
   - Stratified train/test split + Stratified k-fold cross-validation
   - Hyperparameter tuning (GridSearchCV/RandomizedSearchCV) for RF, KNN, SVM
   - Add stronger algorithms: XGBoost, LightGBM, Gradient Boosting, and a simple Neural Network (MLPClassifier)
   - Build a Voting/Stacking ensemble combining the top 3 performers
   - Apply SMOTE or class_weight='balanced' to handle imbalance and boost recall on the anomaly class
   - Feature importance via SHAP or permutation importance (replace the simple correlation heatmap)
3. **Evaluate thoroughly**:
   - Report Accuracy, Precision, Recall, F1, ROC-AUC, and confusion matrix (TP/TN/FP/FN) for every model
   - Cross-validation mean ± std for robustness
   - A clear comparison table: Paper's 3 models vs. new/improved models
   - Highlight % improvement over the paper's Random Forest benchmark
   - Plot ROC curves and a feature importance chart
4. **Explain the reasoning** — why tuning/boosting/balancing should improve results, and discuss accuracy vs. training-time trade-offs.
5. **Output format — also structure the final output as an academic paper extension study**, with these sections:
   - Abstract (3-4 sentences summarizing the improvement achieved)
   - Introduction (brief, referencing the original paper's gap)
   - Methodology (dataset, preprocessing, models, tuning strategy)
   - Results (tables + figures comparing baseline vs. improved models)

- Discussion (why the new approach works better, limitations, future work)
- Conclusion

Deliverables: (a) Clean, well-commented, runnable Python code (assume sklearn, xgboost, lightgbm, imbalanced-learn, shap, matplotlib/seaborn are available), and (b) The academic write-up described above in Markdown.

If an actual CSV dataset isn't available, generate a synthetic but statistically realistic dataset matching the feature descriptions and distributions given above so the code is fully functional end-to-end.